

重 庆 邮 电 大 学

2024 年攻读硕士学位研究生入学考试试题

（回忆修订版）

科目名称： 数据结构（A）卷

科目代码： 802

考生注意事项

- 1、答题前，考生必须在答题纸指定位置上填写考生姓名、报考单位和考生编号，请仔细阅读信封条码粘贴说明。
- 2、所有答案必须写在答题纸上，写在其他地方无效。
- 3、填（书）写必须使用黑色字迹钢笔、圆珠笔或签字笔。
- 4、考试结束，将答题纸和试题一并装入试卷袋中交回。
- 5、本试题满分 150 分，考试时间 3 小时。

一、程序阅读题

1. 有一个整数 n ，其中 n 为 2 的幂。程序通过如下代码计算 $count$ 的值，并返回。请分析并给出该代码的时间复杂度。

```
1. int main() {
2.     int count = 0;
3.     int x = 2;
4.     while (x < n / 2) {
5.         x *= x;
6.         count++;
7.     }
8.     return count;
9. }
```

2. 下面给出一段关于链表操作的代码片段，请将这些代码片段按正确的顺序排列，使其实现将结点 s 插入到结点 p 之前的功能（假设 p 既不是头结点也不是尾结点），并分析时间复杂度。代码片段如下：

- ① $prev \rightarrow next \neq p \ \&\& \ prev \rightarrow next \neq NULL$
- ② $prev \rightarrow next = s$
- ③ $head == NULL \ || \ p == NULL \ || \ s == NULL$
- ④ $prev \rightarrow next == p$

```
1. void insertBefore(Node* head, Node* p, Node* s) {
2.     if (_____) return;
3.     Node* prev = head;
4.     while (_____) {
5.         prev = prev->next;
6.     }
7.     if (_____) {
8.         s->next = p;
9.         _____;
10.    }
11. }
```

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 2 页/共 6 页

3. 合并两个非递减的链表为一个非递增链表，以下是部分代码，请补全代码使其正确实现功能，假设链表的长度分别为 n 和 m ，分析这段代码的时间复杂度。

```

1.  typedef struct Node {
2.      int data;
3.      struct Node* next;
4.  } Node;
5.
6.  Node* mergeAndReverse(Node* La, Node* Lb) {
7.      Node* Lc = NULL; // 非递增链表的头指针
8.      Node* hc = NULL; // 临时指针，用于构建链表
9.
10.     while (____①____) {
11.         Node* p = NULL;
12.         if (____②____) {
13.             p = La;
14.             La = La->next;
15.         } else {
16.             p = Lb;
17.             Lb = Lb->next;
18.         }
19.         ____③____;
20.         ____④____;
21.     }
22.
23.     while (La != NULL) {
24.         Node* p = La;
25.         La = La->next;
26.         p->next = Lc;
27.         Lc = p;
28.     }
29.
30.     while (Lb != NULL) {
31.         Node* p = Lb;
32.         Lb = Lb->next;
33.         p->next = Lc;
34.         Lc = p;
35.     }
36.
37.     return Lc;
38. }

```

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 3 页/共 6 页

4. 请补全下面拓扑排序算法中的代码，假设顶点数为 $|V|$ ，边数为 $|E|$ ，分析这段代码的时间复杂度。

```

1.  bool Topologicalsort(Graph G) {
2.      InitStack(S);
3.      for (int i = 0; i < G.vexnum; i++) {
4.          if (indegree[i] == 0) {
5.              ① _____;
6.          }
7.      }
8.      int count = 0;
9.      while ( ② _____ ) {
10.         ③ _____;
11.         print[count++] = i;
12.         for (p = G.vertices[i].firstarc; ④ _____; p = p->nextarc) {
13.             v = p->adjvex;
14.             if (!(--indegree[v])) {
15.                 Push(S, v);
16.             }
17.         }
18.     }
19.     if (count < G.vexnum) {
20.         return false;
21.     } else {
22.         return true;
23.     }
24. }
```

5. 描述以下递归代码的功能。

```

1.  int func(TreeNode* root) {
2.      if (root == NULL) {
3.          return 0;
4.      }
5.      if (root->left == NULL && root->right == NULL) {
6.          return 1;
7.      }
8.      return func(root->left) + func(root->right);
9.  }
```

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 4 页/共 6 页

二、填空题

1. 设循环队列的表长为 n , f 指向元素的前一个位置, r 指向最后一个元素的位置。该循环队列的长度为_____。

2. 将中缀表达式 $a+b-a*((c+d)/e-f)+g$ 转换为后缀表达式_____。

3. 给定字符串 $abcaabbabcbab$, 求其对应的 $next$ 数组。

	a	b	c	a	a	b	b	a	b	c	a	b
下标 i	1	2	3	4	5	6	7	8	9	10	11	12
$next[i]$												

4. 在哈夫曼树中, 已知有 1247 个结点, 叶子结点的个数为_____。

5. 对序列 $\{1, 2, 3, 4, 5, 6, 7\}$ 进行顺序插入构建平衡二叉树, 平衡因子为 0 的结点个数为_____。

6. 给定一个二维数组 $A[10][15]$, $A[0][0]$ 的地址为 644, 行优先存储, 每个元素占用一个地址空间, $A[4][5]$ 的地址为_____。

7. 已知一棵树有 2011 个结点, 其中 116 个是叶结点, 其对应二叉树中右子树为空的结点数量为_____。

8. 设森林 F 中有三棵树, 第一, 第二, 第三棵树的结点个数分别为 M_1 、 M_2 和 M_3 。与森林 F 对应的二叉树根结点的左子树上的结点个数是_____。

9. 已知一棵完全二叉树的第六层有 8 个叶结点, 树根位于第一层, 树的最多结点数为_____。

三、综合应用题

1. 设数组 $S[] = \{93, 946, 372, 9, 146, 151, 301, 485, 236, 372, 43, 892\}$,
(1) 采用最低位优先(LSD)基数排序将 S 排列成升序序列。写出每一趟排序的结果。

(2) 增量等于 5 时进行希尔排序, 写出第一趟排序结果。

(3) 以第一个元素为轴进行快速排序, 写出第一趟排序结果。

(4) 构造小根堆, 写出调整后的小根堆。

2. 设散列函数 $H(key) = key \text{MOD} 11$, 用线性探测再散列法解决冲突。对关键字序列 $(13, 28, 72, 5, 16, 18, 7, 11, 24)$ 在地址空间为 0-10 的散列区中建散列表, 画出此表, 并求等概率情况下查找成功和查找失败时的平均查找长度。

下标	0	1	2	3	4	5	6	7	8	9	10
关键字											

3. 一棵二叉树的先序、中序和后序序列如下, 其中有部分未标出:

先序序列: C D E G H I K

中序序列: C B F A J K I G

后序序列: E F D B J I H A

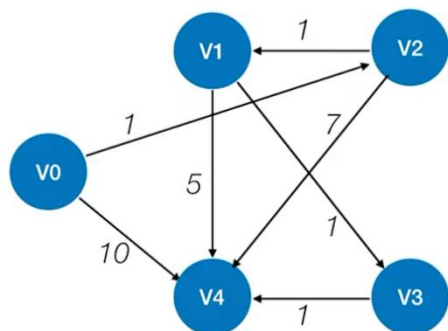
(1) 根据所给序列补全先序、中序、后序序列。

(2) 构造二叉树。

(3) 画出二叉树的中序线索链表。

注: 所有答案必须写在答题纸上, 试卷上作答无效! 第 5 页/共 6 页

4.使用弗洛伊德算法,计算下面图的最短路径, 给出每一步的最短路径矩阵 $D^{(i)}$ 以及保存中转点的矩阵 $path^{(i)}$ 。



$$A^{(-1)} = \begin{matrix} & V_0 & V_1 & V_2 & V_3 & V_4 \\ \begin{matrix} V_0 \\ V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{pmatrix} 0 & \infty & 1 & \infty & 10 \\ \infty & 0 & \infty & 1 & 5 \\ \infty & 1 & 0 & \infty & 7 \\ \infty & \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

$$Path^{(-1)} = \begin{matrix} & V_0 & V_1 & V_2 & V_3 & V_4 \\ \begin{matrix} V_0 \\ V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{pmatrix} -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \end{pmatrix} \end{matrix}$$

5.设某通信信息由若干字母组成, 字母以及其出现的次数如下表所示。采用赫夫曼(Huffman)编码算法对上述序列编码, 给出相应的 Huffman 树, 以及每个字符的 Huffman 码。

字符	a	b	c	d	e	f	g
出现频率	2	5	6	8	13	19	36

6.对["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]的月份集合, 执行以下操作:

- (1) 构造二叉排序树, 并计算查找成功的平均查找长度。
- (2) 对月份简写进行排序后, 利用折半查找, 计算查找成功的平均查找长度。
- (3) 构造二叉平衡树, 并计算成功的平均查找长度。

四、算法设计与分析题

1.编写代码及思想, 判断一棵树是否为二叉排序树, 结构体已给出。

```

1. typedef struct BiTNode {
2.     int data; // 数据域, 存储节点值
3.     struct BiTNode* lchild; // 指向左子树的指针
4.     struct BiTNode* rchild; // 指向右子树的指针
5. } BiTNode, *BiTree;
```

2.编写代码及思想, 判断在无向图中是否存在从顶点 i 到顶点 j 的长度为 k 的路径 (使用邻接表表示图)

3.编写代码及思想, 计算二叉树的繁荣度 (即每一层的结点数乘以该层的层数)。

注: 所有答案必须写在答题纸上, 试卷上作答无效! 第 6 页/共 6 页

重 庆 邮 电 大 学

2025 年攻读硕士学位研究生入学考试试题

(回忆修订版)

科目名称: 数据结构 (A) 卷

科目代码: 802

考生注意事项

- 1、答题前, 考生必须在答题纸指定位置上填写考生姓名、报考单位和考生编号, 请仔细阅读信封条码粘贴说明。
- 2、所有答案必须写在答题纸上, 写在其他地方无效。
- 3、填(书)写必须使用黑色字迹钢笔、圆珠笔或签字笔。
- 4、考试结束, 将答题纸和试题一并装入试卷袋中交回。
- 5、本试题满分 150 分, 考试时间 3 小时。

注: 所有答案必须写在答题纸上, 试卷上作答无效! 第 1 页/共 6 页

一、程序阅读题

1、求以下代码的时间复杂度，并分析原因。

```
1. void move(char a, char b, int n) {
2.     printf("将第%d 个盘子从%c 移动到%c", n, a, b);
3.     printf("%s\n", buffer);
4. }
5.
6. void F(int n, char a, char b, char c) {
7.     if (n == 1) {
8.         move(a, c, n);
9.     } else {
10.        F(n - 1, a, c, b);
11.        move(a, c, n);
12.        F(n - 1, b, a, c);
13.    }
14. }
```

2、下列代码交换双向链表中 p 结点和 p 结点的前驱。请填写完整代码。

双向链表存储结构：

```
1. struct DLinkNode{
2.     ElementType data;
3.     Position prior; //Front pointer
4.     Position next; //Rear pointer
5. };
6. struct DLinkNode;

1. void SwapWithPrevious(DLinkNode* p) {
2.     DLinkNode* q = p->prior;
3.     _____①_____;
4.     p->next = q;
5.     q->prior = p;
6.     _____②_____;
7.     p->prior->next = p;
8.     q->next->prior = q;
9. }
```

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 1 页/共 6 页

3、下列代码实现循环单链表队列的出队入队，请填写完整代码。
队列的链式存储类型可以描述为：

```
1. typedef struct LinkNode {
2.     ElemType data;
3.     struct LinkNode *next;
4. } LinkNode;
5.
6. typedef struct {
7.     LinkNode *rear;
8. } LinkQueue;
```

```
1. void EnQueue(LinkQueue *Q, LinkNode *p) {
2.     if (Q->rear == NULL) {
3.         Q->rear = p;
4.         p->next = p;
5.     } else {
6.         _____ ①;
7.         Q->rear->next = p;
8.         Q->rear = p;
9.     }
10. }
```

```
1. LinkNode* DeQueue(LinkQueue *Q) {
2.     LinkNode *p;
3.
4.     if (Q->rear == NULL) {
5.         return NULL;
6.     }
7.
8.     p = Q->rear->next;
9.     if (Q->rear == p) {
10.        _____ ②;
11.    } else {
12.        Q->rear->next = p->next;
13.    }
14.
15.    return p;
16. }
```

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 1 页/共 6 页

4、下列代码实现带头结点的单链表的简单选择排序，请填写完整代码。

单链表存储结构：

```
1. typedef struct LNode {
2.     ElemType data;
3.     struct LNode* next;
4. }LNode,*LinkList;

1. void SelectionSort(LinkList &L) {
2.     LNode *p, *q, *min;
3.     ①;
4.
5.     while (p != NULL) {
6.         min = p;
7.         q = p->next;
8.
9.         while (q != NULL) {
10.            if (q->data < min->data) {
11.                min = q;
12.            }
13.            q = q->next;
14.        }
15.
16.        if (②) {
17.            ElemType temp = p->data;
18.            p->data = min->data;
19.            min->data = temp;
20.        }
21.
22.        ③;
23.    }
24. }
```

二、填空题

1、分析这段代码在给定输入 $a[] = \{5, 4, 3, 2, 1\}$ 的时间复杂度_____

```
void F(int a[], int i, int n) {
    if (i >= n - 1) {
        return;
    }
    for (int j = i; j < n - 1; j++) {
        if (a[j] > a[j + 1]) {
```

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 1 页/共 6 页

```

        int temp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = temp;
    }
}
F(a, i + 1, n);

```

- 2、元素 1,1,2,3,4 依次入栈，以 4 为开头的出栈序列有____个。
- 3、已知数组 $A_{9 \times 3 \times 5 \times 8}$ ，其中 A_{0000} 的地址为 100，每一个整数占 4B，则 A_{2147} 的地址为_____。
- 4、哈夫曼树有 199 个结点，则叶子结点有____个。
- 5、对于后缀表达式 1 2 4 2 / + 5 * -，经过运算后的值为_____。
- 6、森林 F 有 15 条边，25 个结点，则森林中一共有____棵树。
- 7、getHead[getTail[((a,b),(c,d))]]=_____。
- 8、利用 KMP 算法，求 "abcaaba" 的 next 数组_____。
- 9、在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树中叶节点的个数为_____。
- 10、如图所示，有一个小根堆，当插入 3 后，小根堆经过调整后为_____。

三、综合应用题

- 1、已知一棵二叉查找树/二叉排序树(BST)的先序遍历序列为 23, 15, 12, 18, 56, 29, 34, 71。

- (1) 请画出此二叉树的形态。
- (2) 请给出这棵树的后序遍历序列。
- (3) 画出此二叉树的中序线索二叉树。
- (4) 画出与此二叉树对应的森林。

- 2、设散列表为 HT[0..12]，即表的大小为 $m=13$ 。现采用双散列法解决冲突。散列函数和再散列函数分别为：

$$H_0(\text{key}) = \text{key} \% 13$$

$$H_i = (H_{i-1} + (\text{reverse}(x) + 1) \% 11 + 1) \% 13$$

其中 reverse(x) 是将 x 逆置，如：reverse(789)=987。

- (1) 若输入的关键码为 2, 8, 31, 20, 70, 59, 25, 28，请画出插入这 8 个关键码后得到的散列表。

- (2) 计算查找成功 ASL。

哈希地址	0	1	2	3	4	5	6	7	8	9	10	11	12
关键字													
探测次数													

注：所有答案必须写在答题纸上，试卷上作答无效！ 第 1 页/共 6 页

3、给定字符 a 到 g，它们的权值分别如下：

a: 5 b: 9 c: 12 d: 13 e: 16 f: 45 g: 50

(1) 请根据给定的字符及其对应的权值，使用哈夫曼编码算法构建一棵哈夫曼树。要求在每次合并两个结点时，确保左结点的权值小于等于右结点的权值。

(2) 根据哈夫曼树的结构，写出每个字符的哈夫曼编码，要求左结点为 0，右结点为 1，填出下表中的对应编码。

字符	权值	编码
a	5	
b	9	
c	12	
d	13	
e	16	
f	45	
g	50	

4、已知有序数组为[7, 15, 23, 35, 41, 53, 62, 70, 81, 94]，对该数组进行二分查找，回答以下问题：

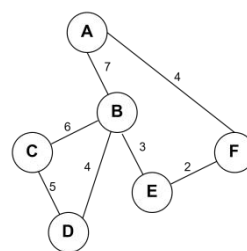
- (1) 画出二叉查找树；
- (2) 查找 53 需要找哪些元素；
- (3) 查找 90 需要找哪些元素；
- (4) 计算查找成功和查找失败的平均查找长度。

5、已知带权的完全无向图 G，回答以下问题：

(1) 从 A 开始，分别写出图 G 的广度优先搜索 (BFS) 和深度优先搜索 (DFS) 序列。如果在遍历过程中有多个相同权值的路径，优先选择字母顺序较小的结点。

(2) 使用 Prim 算法从结点 A 开始构建最小生成树。如果在选择路径时存在多个相同权值的路径，优先选择字母顺序较小的结点。

(3) 使用 Kruskal 算法构建最小生成树。如果 X 在选择路径时存在多个相同权值的路径，优先选择字母顺序较小的结点。

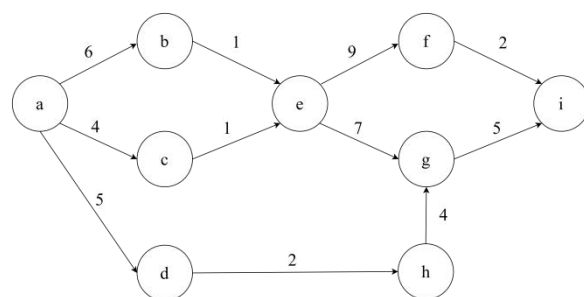


6、给定一个 AOE 网，请回答以下问题：

(1) 计算每个事件的最早发生时间 (ve) 和最晚发生时间 (vl)；

(2) 计算每个活动的最早开始时间 (e) 和最迟完成时间 (l)；

(3) 确定关键路径。



四、程序设计题

1、给定一个单链表，找到倒数第 k 个元素。如果找到该元素，打印该元素的值并返回 1；否则返回 0。

(1) 描述算法的基本设计思想；

(2) 根据设计思想，采用 C 语言描述算法。

结点定义如下：

```
typedef struct node {
    int data;
    struct node *next;
} LNode, *LinkList;
```

2、给定一棵二叉树，输出从根结点到叶子结点的最长路径，打印路径的结点值，并输出该路径的长度。

(1) 描述算法的基本设计思想；

(2) 根据设计思想，采用 C 语言描述算法。

二叉树的链式存储结构描述如下：

```
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
```

3、给定一个邻接表表示的连通图，输出从顶点 u 到顶点 v 并经过顶点 k 的一条路径。如果不存在这样的路径，输出 -1。

(1) 描述算法的基本设计思想；

(2) 根据设计思想，采用 C 语言描述算法。

```
#define MaxVertexNum 100
typedef char InfoType;    //附加信息类型
typedef int VertexType;   //顶点数据类型
typedef struct ArcNode{
    int adjvex;            //该弧所指向的顶点位置
    struct ArcNode *next; //下个结点
    //InfoType info;      //当前结点（弧）的信息
} ArcNode;
typedef struct VNode{
    VertexType data; //顶点信息
    ArcNode *first; //指向第一个依附于该顶点弧的指针
} VNode, AdjList[MAX_VERTEX_NUM];
typedef struct {
    AdjList vertices;
    int vexnum, arcnum; //图的当前顶点数和弧数
} ALGraph;
```

重庆邮电大学考研

2024 年 802 真题答案与解析

一、程序阅读题

1. 循环终止的条件是 $x \geq n/2$ 。所以，循环会运行直到 $2^{2^k} \geq n/2$ 。为了找到

最大迭代次数 k ，我们需要解决不等式：

$$2^{2^k} \geq \frac{n}{2}$$

取对数，我们有：

$$2^k \geq \log_2\left(\frac{n}{2}\right) = \log_2 n - 1$$

因此， $k \geq \log_2(\log_2 n - 1)$ 近似为 $\log_2(\log_2 n)$ 。因此，循环的迭代次数 k 是对数级别的，即 $\log_2(\log_2 n)$ 。

时间复杂度为： $O(\log_2(\log_2 n))$

2. ③①④②

3.

① $La \neq \text{NULL} \ \&\& \ Lb \neq \text{NULL}$

② $La \rightarrow \text{data} \leq Lb \rightarrow \text{data}$

③ $p \rightarrow \text{next} = Lc$

④ $Lc = p$

算法的时间复杂度为 $O(n + m)$ ，其中 n 和 m 分别是链表 La 和 Lb 的长度。

解析：该代码首先遍历两个链表 La 和 Lb ，按元素值从小到大的顺序比较两个链表的结点，将较小值的结点插入到 Lc 链表头部，从而实现逆序合并。之后，处理剩余的未遍历完的结点，并继续插入到 Lc 的头部。最终返回合并后的非递增链表 Lc 。

4.

① $\text{Push}(S, i)$

②!IsEmpty(S)

③Pop(S,i)

④p!=NULL

该算法的时间复杂度为 $O(|V|+|E|)$

解析:

- 入度计算: 首先, 通过遍历每个顶点的邻接链表来计算每个顶点的入度。
- 初始化栈: 然后, 将所有入度为 0 的顶点压入栈中。
- 拓扑排序: 通过循环, 弹出栈顶元素, 将其输出, 并将该顶点所指向的顶点的入度减 1。如果某个顶点的入度减为 0, 则将其压入栈中。
- 回路检测: 最后, 如果输出的顶点数小于图中的顶点数, 则说明图中存在回路, 拓扑排序失败; 否则, 排序成功。

5.该代码实现了递归计算二叉树中叶结点数量的功能。

二、填空题

1. $(r-f+n)\%n$

解析: 循环队列的长度计算公式如下: 长度 = $(\text{rear} - \text{front} + \text{maxsize}) \% \text{maxsize}$ 其中, rear 表示队尾指针, front 表示队头指针, maxsize 表示队列的最大长度

2. $ab+acd+e/f-*-g+$

解析:

待处理序列	栈	后缀表达式	当前扫描元素	动作
$a+b-a*((c+d)/e-f)+g$			a	a 加入后缀表达式
$+b-a*((c+d)/e-f)+g$		a	+	+入栈
$b-a*((c+d)/e-f)+g$	+	a	b	b 加入后缀表达式
$-a*((c+d)/e-f)+g$	+	ab	-	+出栈, -入栈

$a*((c+d)/e-f)+g$	-	ab+	a	a 加入后缀表达式
$*((c+d)/e-f)+g$	-	ab+a	*	*入栈
$((c+d)/e-f)+g$	-*	ab+a	(	(入栈
$(c+d)/e-f)+g$	-(	ab+a	(	(入栈
$c+d)/e-f)+g$	-*((	ab+a	c	c 加入后缀表达式
$+d)/e-f)+g$	-*((	ab+ac	+	+入栈, +加入后缀表达式
$d)/e-f)+g$	-*((+	ab+ac	d	d 加入后缀表达式
$) /e-f)+g$	-*((+	ab+acd	)	+(依次出栈, +加入后缀表达式
$/e-f)+g$	-(	ab+acd+	/	/入栈
$e-f)+g$	-(/	ab+acd+	e	e 加入后缀表达式
$-f)+g$	-(/	ab+acd+e	-	/出栈, /加入后缀表达式, -入栈
$f)+g$	-(	ab+acd+e/	f	f 加入后缀表达式
$) +g$	-(	ab+acd+e/f	)	-(依次出栈, -加入后缀表达式
$+g$	-*	ab+acd+e/f-	+	*-依次出栈, 加入后缀表达式, +入栈
g	+	ab+acd+e/f-*-	g	g 加入后缀表达式
	+	ab+acd+e/f-*--g		扫描完毕, 运算符依次退栈加入表达式
		ab+acd+e/f-*--g+		完成

3.

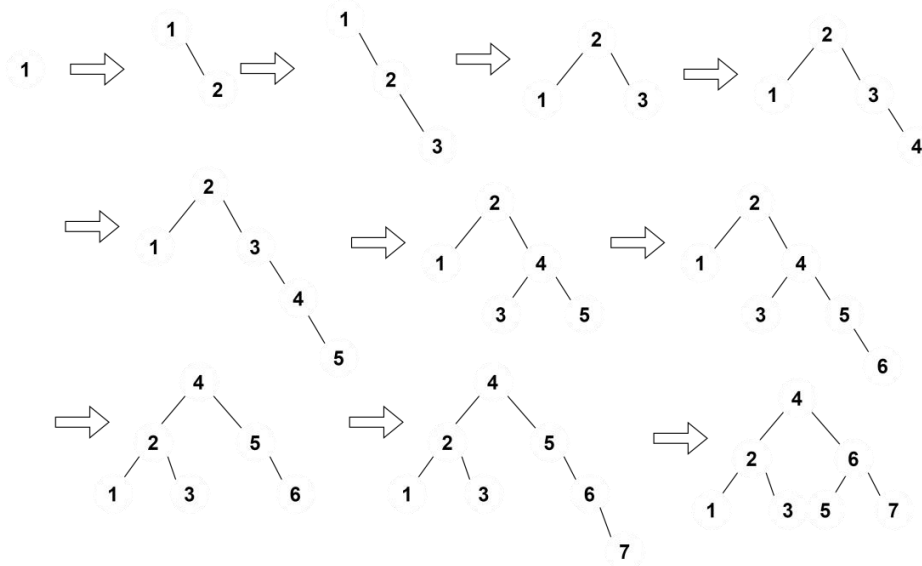
	a	b	c	a	a	b	b	a	b	c	a	b
下标 i	1	2	3	4	5	6	7	8	9	10	11	12
next[i]	0	1	1	1	2	2	3	1	2	3	4	5

4. 624

解析：在哈夫曼树中，只有度为 0 和 2 的结点，又因为 $N_0=N_2+1$ ，可以得到总结点 $N=2N_0-1=1247$ ，可算得 $N_0=624$ 。

5. 7

解析：构造过程如下：



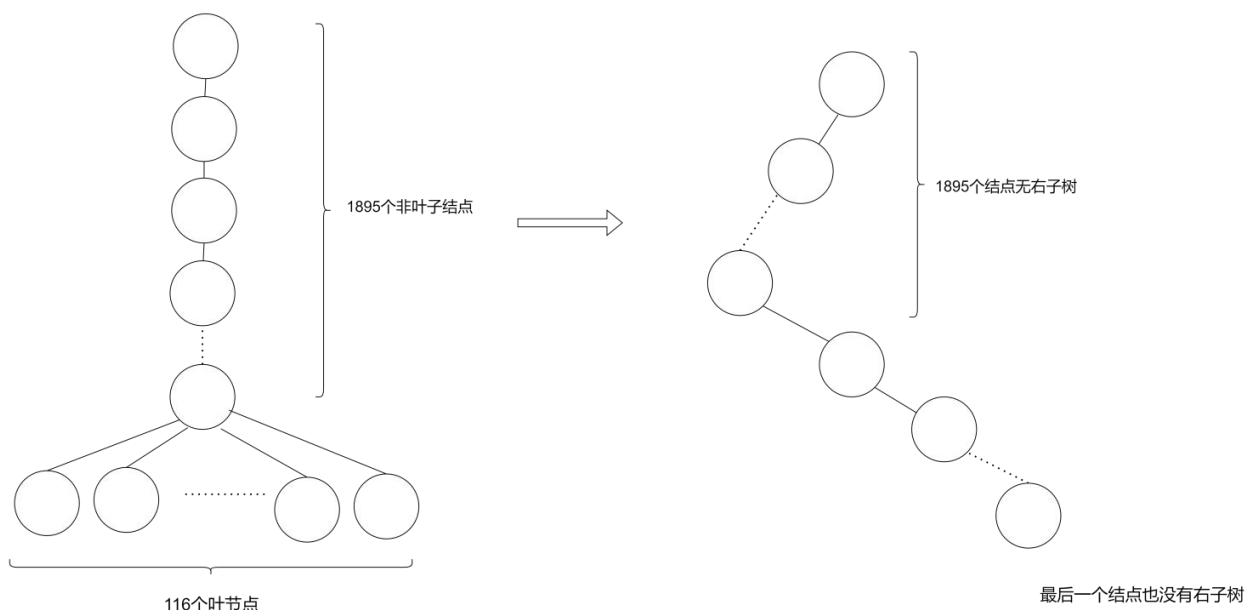
6. 709

解析： $LOC(A[i][j]) = LOC(A[0][0]) + ((i \times \text{列数}) + j) \times \text{元素大小}$

所以 $LOC(A[4][5]) = 644 + ((4 \times 15) + 5) \times 1 = 709$

7. 1896

解析：特殊法



8. M1-1

解析：将森林 F 中的每棵树都视为一个独立的二叉树。对于每棵树，选择其中的一棵树根结点作为二叉树的根结点，而其他结点则成为这个根结点的左子树的结点。这样，每棵树都被转化为一个二叉树，并且它们的根结点都成为了原二叉树的左子树的结点。最后，将这些二叉树合并成一个大的二叉树，其中根结点是森林 F 中第一棵树的根结点，其左子树包括了森林 F 中其他树的根结点和它们的左子树。根据这个转化，与森林 F 对应的二叉树根结点的左子树上的结点个数是 M1-1。

9. 111

解析：完全二叉树比满二叉树只是在最下面一层的右边缺少了部分叶结点,而最后一层之上是个满二叉树,并且只有最后两层有叶结点。第 6 层有叶结点则完全二叉树的高度可能为 6 或 7,显然树高为 7 时结点更多。若第 6 层上有 8 个叶结点,则前六层为满二叉树,而第 7 层缺失了 $8 \times 2 = 16$ 个叶结点,故完全二叉树的结点个数最多为 $(2^7 - 1) - 16 = 111$ 个结点。

三、综合应用题

1.

(1)

第 1 趟排序（按个位排序）

$S = \{151, 301, \underline{372}, 372, 892, 093, 043, 485, 946, 146, 236, 009\}$

第2趟排序（按十位排序）

$S = \{301, 009, 236, 043, 946, 146, 151, \underline{372}, 372, 485, 892, 093\}$

第3趟排序（按百位排序）

$S = \{009, 093, 043, 146, 151, 236, 301, \underline{372}, 372, 485, 892, 946\}$

(2)

$S[] = \{43, 301, \underline{372}, 9, 146, 93, 892, 485, 236, 372, 151, 946\}$

(3)

$S[] = \{43, 9, \mathbf{93}, \underline{372}, 146, 151, 301, 485, 236, 372, 946, 892\}$

(4)

$S[] = \{9, 43, 151, 236, 93, \underline{372}, 301, 485, 946, 372, 146, 892\}$

2.

下标	0	1	2	3	4	5	6	7	8	9	10
关键字	11		13	24		5	28	72	16	18	7
成功比较次数	1		1	2		1	1	2	4	3	4
失败比较次数	2	1	3	2	1	8	7	6	5	4	3

查找成功 $ASL = \text{各元素冲突之和} / \text{关键字个数}$

查找失败 $ASL = \text{查找各地址失败的比较次数之和} / \text{mod 的长度}$

$$ASL_{\text{成功}} = (1+1+2+1+3+3+4+1+2)/9 = 2$$

$$ASL_{\text{失败}} = (2+1+3+2+1+8+7+6+5+4+3)/11 = 42/11$$

3.

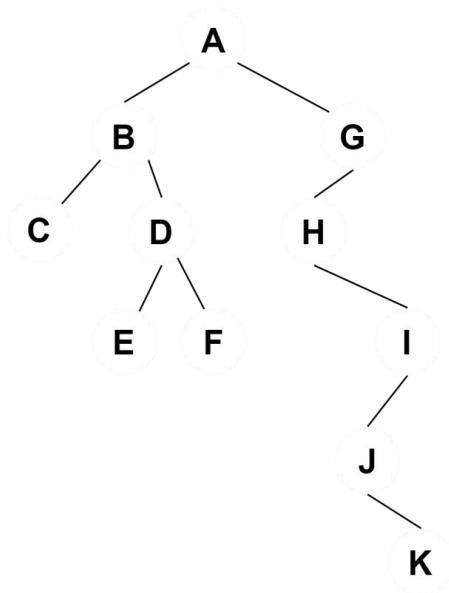
(1)

先序序列: A B C D E F G H I J K

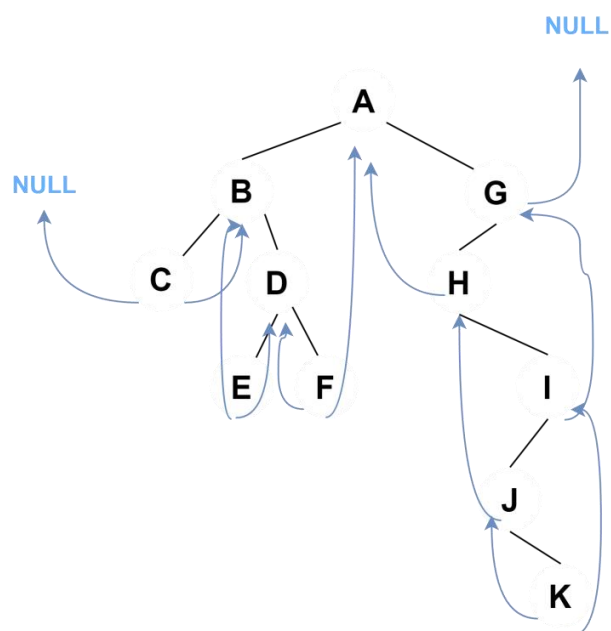
中序序列: CBEDFAHJKIG

后序序列: CEFDBKJIHGA

(2)

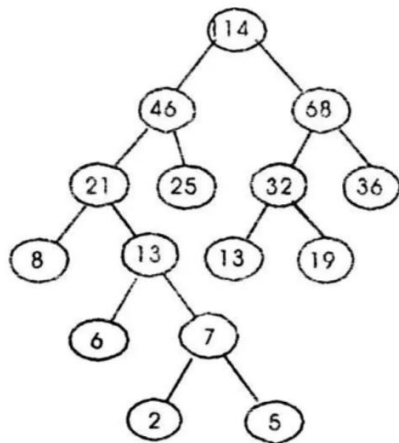


(3)



	V_0	V_1	V_2	V_3	V_4		V_0	V_1	V_2	V_3	V_4	
$A^{(0)} =$	V_0	0	∞	1	∞	10	V_0	-1	-1	-1	-1	-1
	V_1	∞	0	∞	1	5	V_1	-1	-1	-1	-1	-1
	V_2	∞	1	0	∞	7	V_2	-1	-1	-1	-1	-1
	V_3	∞	∞	∞	0	1	V_3	-1	-1	-1	-1	-1
	V_4	∞	∞	∞	∞	0	V_4	-1	-1	-1	-1	-1
	V_0	V_1	V_2	V_3	V_4		V_0	V_1	V_2	V_3	V_4	
$A^{(1)} =$	V_0	0	∞	1	∞	10	V_0	-1	-1	-1	-1	-1
	V_1	∞	0	∞	1	5	V_1	-1	-1	-1	-1	-1
	V_2	∞	1	0	2	6	V_2	-1	-1	-1	1	1
	V_3	∞	∞	∞	0	1	V_3	-1	-1	-1	-1	-1
	V_4	∞	∞	∞	∞	0	V_4	-1	-1	-1	-1	-1
	V_0	V_1	V_2	V_3	V_4		V_0	V_1	V_2	V_3	V_4	
$A^{(2)} =$	V_0	0	2	1	3	7	V_0	-1	2	-1	2	2
	V_1	∞	0	∞	1	5	V_1	-1	-1	-1	-1	-1
	V_2	∞	1	0	2	6	V_2	-1	-1	-1	1	1
	V_3	∞	∞	∞	0	1	V_3	-1	-1	-1	-1	-1
	V_4	∞	∞	∞	∞	0	V_4	-1	-1	-1	-1	-1
	V_0	V_1	V_2	V_3	V_4		V_0	V_1	V_2	V_3	V_4	
$A^{(3)} =$	V_0	0	2	1	3	4	V_0	-1	2	-1	2	3
	V_1	∞	0	∞	1	2	V_1	-1	-1	-1	-1	3
	V_2	∞	1	0	2	3	V_2	-1	-1	-1	1	3
	V_3	∞	∞	∞	0	1	V_3	-1	-1	-1	-1	-1
	V_4	∞	∞	∞	∞	0	V_4	-1	-1	-1	-1	-1
	V_0	V_1	V_2	V_3	V_4		V_0	V_1	V_2	V_3	V_4	
$A^{(4)} =$	V_0	0	2	1	3	4	V_0	-1	2	-1	2	3
	V_1	∞	0	∞	1	2	V_1	-1	-1	-1	-1	3
	V_2	∞	1	0	2	3	V_2	-1	-1	-1	1	3
	V_3	∞	∞	∞	0	1	V_3	-1	-1	-1	-1	-1
	V_4	∞	∞	∞	∞	0	V_4	-1	-1	-1	-1	-1

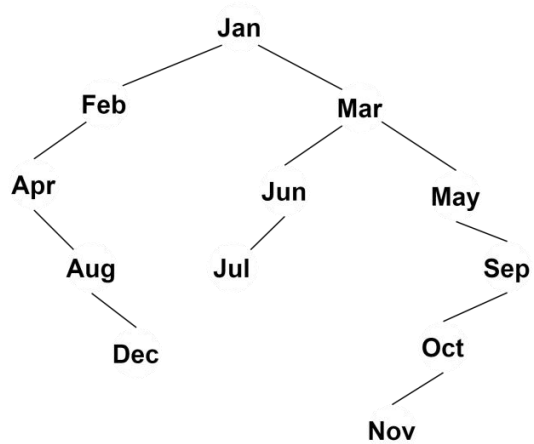
5.



字符	a	b	c	d	e	f	g
出现频率	2	5	6	8	13	19	36
哈夫曼编码	00110	00111	0010	000	100	101	11

6.

(1)



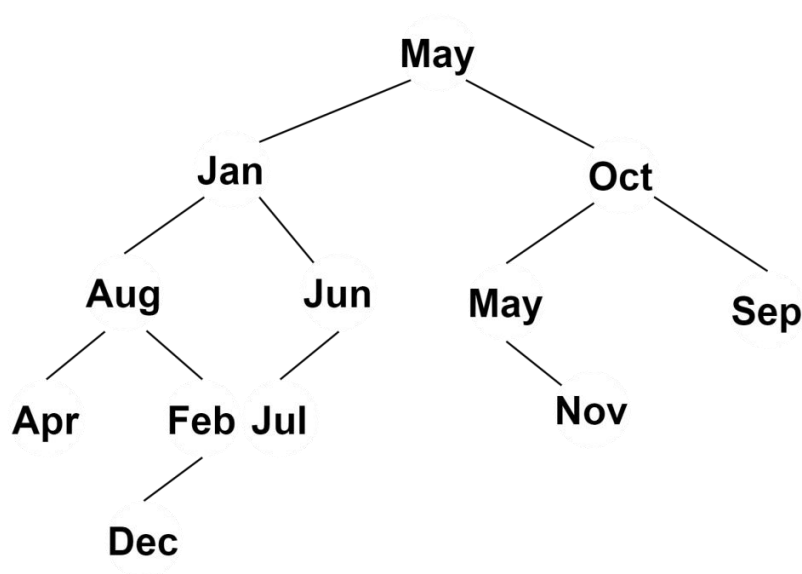
$$ASL = (6 + 5 \times 2 + 4 \times 3 + 3 \times 3 + 2 \times 2 + 1) / 12 = 7/2$$

(2)

排序后: ["Apr", "Aug", "Dec", "Feb", "Jan", "Jul", "Jun", "Mar", "May", "Nov", "Oct", "Sep"]

$$ASL = (4 \times 5 + 3 \times 4 + 2 \times 2 + 1 \times 1) / 12 = 37 / 12$$

(3)



$$ASL = (5 \times 1 + 4 \times 4 + 3 \times 4 + 2 \times 2 + 1 \times 1) / 12 = 19 / 6$$

四、算法设计与分析题

1、思想：对于一棵二叉排序树，任意节点的左子树中的所有节点的值都小于该节点的值，而右子树中的所有节点的值都大于该节点的值。按照如下的算法就可以递归地检查整个树：

递归检查：

- 对于每个结点，判断其值是否在允许的范围内。
- 初始情况下，对于根结点，允许的范围是 $-\infty$ 到 ∞ 。
- 对于左子树，值的范围更新为 $-\infty$ 到当前结点的值。
- 对于右子树，值的范围更新为当前结点的值到 ∞ 。
- 递归地检查整个树。

代码:

```
1. // 判断二叉树是否为二叉排序树的辅助函数
2. bool isBSTUtil(BiTNode* t, int min, int max) {
3.     // 空树是二叉排序树
4.     if (t == NULL)
5.         return true;
6.
7.     // 检查当前节点是否在允许的范围内
8.     if (t->data <= min || t->data >= max)
9.         return false;
10.
11.    // 递归检查左子树和右子树
12.    return isBSTUtil(t->lchild, min, t->data) &&
13.           isBSTUtil(t->rchild, t->data, max);
14. }
15.
16. // 判断二叉树是否为二叉排序树
17. bool isBST(BiTree T) {
18.     return isBSTUtil(T, INT_MIN, INT_MAX);
19. }
```

2、思想：代码通过递归的方法来检查无向图中是否存在从顶点 i 到顶点 j 且长度为 k 的路径。它首先标记当前顶点 i 为已访问，然后递归地遍历与顶点 i 相邻的所有顶点。在每次递归中，路径长度 k 减一。如果找到从某个邻接顶点到达目标顶点 j 且剩余路径长度为零的路径，则返回 1 表示路径存在；否则，递归结束时将当前顶点的访问状态重置为未访问，继续检查其他可能的路径。如果所有路径都不满足条件，最终返回 0，表示不存在符合条件的路径。

代码:

```
1. int visited[MAXSIZE];
2. int exist_path_len(ALGraph G, int i, int j, int k){
3.     if(i==j&& k==0){
4.         return 1;
5.     }
6.     else if(k>0){
7.         visited[i]=1;
8.         for(ArcNode* p=G.vertices[i].firstarc;p;p=p->nextarc){
9.             int temp=p->adjvex;
10.            if(!visited[temp]&& exist_path_len(G,temp,j,k-1)){
11.                return 1;
12.            }
13.            visited[i]=0;
```

```

14.     }
15.     return 0;
16. }

```

3、思想：利用二叉树层次遍历的思想，在遍历过程中加一个标志每一层的层数 level 以及每一层结点个数的变量 levelSize, 层次遍历结束后，就得到了繁荣度。

代码：

```

1. int calculateProsperity(BiTree T) {
2.     if (!T) return 0;
3.     Queue Q;
4.     InitQueue(Q);
5.     int last=1;
6.     int level = 1;
7.     int totalProsperity = 0;
8.     BiTNode *p=T;
9.     EnQueue(Q,p);
10.    int levelSize=0;
11.    while (!IsEmpty(Q)) {
12.        p=DeQueue(Q);
13.        levelSize++;
14.        if (p->lchild) EnQueue(Q, p->lchild);
15.        if (p->rchild) EnQueue(Q, p->rchild);
16.
17.        // 计算当前层的繁荣度，更新 last
18.        if(Q.front==last){
19.            totalProsperity += levelSize * level;
20.            level++;
21.            last=Q.rear;
22.            levelSize=0;
23.        }
24.    }
25.
26.    FreeQueue(q);
27.    return totalProsperity;
28. }

```

22、

排序题原始序列: 1 5 4 6 3 8 2 7

- (1) 冒泡排序: 1 5 2 4 6 3 8 7
- (2) 直接插入排序: 1 5 4 6 3 8 2 7
- (3) 快速排序: 2 1 4 3 5 8 6 7
- (4) 简单选择排序: 1 5 4 6 3 8 2 7
- (5) 二路归并排序: 1 5 4 6 3 8 2 7

四、 算法分析与设计题(本大题共2小题, 共30分)

23、

```
1 void PatternInvertL(LinkList L, int i, int m) {
2     if(i <= 1 || i >= m || m <= 3) return; // 非法参数
3     // 找到第i-1个结点
4     LNode *p = L;
5     for(int j = 1; j <= i-1; ++j) {
6         p = p->next;
7     }
8
9     // 从i到m的结点都头插到p之后
10    LNode *q = p->next,
11    *end = p->next; // 第i个结点会作为反转后的新尾结点
12    LNode *aux;
13    for(int j = i; j <= m; ++j) {
14        aux = q->next; // 暂存q的后继
15        q->next = p->next; // 头插q之后
16        p->next = q;
17        q = aux; // 处理下一个结点
18    }
19    // end的next指向p的next, 保持循环
20    end->next = p->next;
21 }
```

24、

```
1  #include <stdio.h>
2  #define MaxSize 150
3
4  int ans[MaxSize][MaxSize];    // 存储结果
5
6  void showAns(int g[][MaxSize], int n, int m) {
7      for(int i = 0; i < n; ++i) {
8          for(int j = 0; j < m; ++j) {
9              printf("%d ", g[i][j]);
10             }
11             printf("\n");
12         }
13     }
14
15     // 寻找离 i1, j1 最近的白色像素点的距离
16     int getMinDis(char g[][MaxSize], int n, int m, int i1, int j1) {
17         int minDis = INT_MAX;    // 初始化为正无穷
18         for(int i = 0; i < n; ++i)
19             for(int j = 0; j < m; ++j) {
20                 // 白色像素点
21                 if(g[i][j] == '1') {
22                     // 计算最近距离
23                     minDis = min(minDis, (i1 - i >= 0 ? i1 - i : i - i1) + (j1 - j >= 0 ? j1 - j : j - j1));
24                 }
25             }
26         return minDis;
27     }
28
29     // g:像素点; 规模:n*m
30     void minwhite(char g[][MaxSize], int n, int m) {
31         memset(ans, 0, sizeof(ans));    // 全部初始化为0
32         for(int i1 = 0; i1 < n; ++i1) {
33             for(int j1 = 0; j1 < m; ++j1) {
34                 // 当前像素点(i1, j1)
35                 // 黑色像素, 寻找离它最近的白色像素点
36                 if(g[i1][j1] == '0') {
37                     ans[i1][j1] = getMinDis(g, n, m, i1, j1);
38                 }
39             }
40         }
41     }
```

2025 年 802 真题答案与解析

一、程序阅读题

1、答案： $O(2^n)$ 解析：汉诺塔问题的时间复杂度是 $O(2^n)$ ，这是因为该问题采用递归算法来解决，每一次递归都会将问题规模减小。具体来说，当你尝试将 n 个盘子从源柱子移动到目标柱子时，首先需要将 $n-1$ 个盘子从源柱子移动到辅助柱子，然后将第 n 个盘子从源柱子直接移动到目标柱子，最后再将 $n-1$ 个盘子从辅助柱子移动到目标柱子。每一次递归调用都会减少一个盘子的数量，从而将问题规模缩小。这样，每次递归都需要对 $n-1$ 个盘子进行相同的操作，而这个递归调用的层数会一直达到 n ，导致总的操作数是指数级的。具体来说，递归树的高度为 n ，而每一层递归的操作数量大约是 2^{n-1} ，因此总的时间复杂度是 $O(2^n)$ 。（可以试试画一棵递归树）

2、

答案与解析（注释就是解析）：

```
1. void SwapWithPrevious(DLinkNode* p) {
2.     DLinkNode* q = p->prior; // 获取 p 的前驱结点
3.
4.     // 1. 更新 p 和 q 的前驱和后继指针
5.     p->prior = q->prior;      // p 的前驱指向 q 的前驱
6.     p->next = q;              // p 的后继指向 q
7.     q->prior = p;             // q 的前驱指向 p
8.     q->next = p->next;        // q 的后继指向 p 的后继
9.
10.    // 2. 更新 p 的前驱结点和 q 的后继结点
11.    p->prior->next = p;        // p 的前驱结点的后继指向 p
12.    q->next->prior = q;        // q 的后继结点的前驱指向 q
13. }
```

3、

答案与解析（注释就是解析）：

①p->next = Q->rear->next

②Q->rear = NULL

```
1. // 入队操作
2. void EnQueue(LinkQueue *Q, LinkNode *p) {
3.     if (Q->rear == NULL) { // 如果队列为空
4.         Q->rear = p;
5.         p->next = p; // 新结点的 next 指向自己，形成循环
6.     } else {
7.         p->next = Q->rear->next; // 新结点的 next 指向队头
8.         Q->rear->next = p; // 将尾结点的 next 指向新结点
9.         Q->rear = p; // 更新尾指针
10.    }
11. }
```

```
1. // 出队操作
2. LinkNode* DeQueue(LinkQueue *Q) {
3.     LinkNode *p;
4.
5.     if (Q->rear == NULL) { // 队列为空
6.         return NULL;
7.     }
8.
9.     p = Q->rear->next; // 获取队头结点
10.    if (Q->rear == Q->rear->next) { // 队列中只有一个元素
11.        Q->rear = NULL; // 将队列置为空
12.    } else {
13.        Q->rear->next = p->next; // 更新尾结点的 next 指向新的队头
14.    }
15.
16.    return p; // 返回出队的结点
17. }
```

4、

答案与解析（注释就是解析）：

①p = L->next

②min != p

③p = p->next

```

1. void SelectionSort(LinkList &L) {
2.     LNode *p, *q, *min;
3.     p = L->next; // 第一个空: p = L->next, p 指向第一个有效元素
4.
5.     while (p != NULL) {
6.         min = p; // min 最开始指向 p
7.         q = p->next; // q 从 p 的下一个结点开始
8.
9.         // 进行 while 循环, 找到最小值
10.        while (q != NULL) {
11.            if (q->data < min->data) {
12.                min = q; // 更新 min 为更小的元素
13.            }
14.            q = q->next; // 向后移动 q
15.        }
16.
17.        // 第二个空: 若 min 不等于 p, 交换它们的值
18.        if (min != p) {
19.            ElemType temp = p->data;
20.            p->data = min->data;
21.            min->data = temp; // 交换 p 和 min 的值
22.        }
23.
24.        p = p->next; // 第三个空: p 向后移动
25.    }
26.}

```

二、填空题

1、答案: $O(n^2)$ 解析: 这段代码实现了一个递归版本的冒泡排序。在最坏的情况下, 冒泡排序需要进行 $O(n^2)$ 次比较和交换。

2、答案: 9 解析: 首先, 4 是首先出栈的元素。剩下的元素 1, 1, 2, 3 可以按栈的 LIFO 特性进行任意的出栈操作, 且要求在每次出栈时, 栈顶的元素必须先出。而出栈序列个数可以通过 **Catalan 数** 来计算 (详情可以参考王道辅导书栈的部分), 此时 $n=4$, $\frac{1}{n+1}C_{2n}^n = \frac{1}{4+1}C_8^4 = \frac{1}{5} \cdot \frac{8 \cdot 7 \cdot 6 \cdot 5}{1 \cdot 2 \cdot 3 \cdot 4} = 14$, 但注意到有 2 个 1, 如果不做区分的话, 有的出栈序列是重复的, 对于 1, 1, 2, 3 的入栈顺序, 如果对 1 不做区分, 出栈序列会有重复的, 比如 1123 和 1123, 1132 和 1132, 1321 和 1321, 1213 和 1213, 1231 和 1231。只有 2113, 2311, 2131, 3211 是唯一的。所以剔除掉重复的, 只有 $14-5=9$ 种出栈序列。

3、答案：1376

解析：对于四维数组 $A[n_1][n_2][n_3][n_4]$ ，一个元素的大小为 M ，则 $A[i][j][k][l]$ 在内存中的地址可以通过以下公式计算：

$$\text{address}(A[i][j][k][l]) = \text{address}(A[0][0][0][0]) + (i \times n_2 n_3 n_4 + j \times n_3 n_4 + k \times n_4 + l) \times M$$

对于 $A[2][1][4][7]$ ：

$$\text{Address}(A[2][1][4][7]) = 100 + (2 \times 3 \times 5 \times 8 + 1 \times 5 \times 8 + 4 \times 8 + 7) \times 4 = 1376$$

4、答案：100

解析：在哈夫曼树中，存在一个重要的性质：对于一棵哈夫曼树，叶子结点的个数与非叶子结点的个数之间有如下关系：

哈夫曼树有 n 个叶子结点时，总结点数为 $2n - 1$

所以 $2n - 1 = 199$ 可以推出 $n = 100$

5、答案：-19

解析：

待处理序列	栈	当前扫描元素	动作
1 2 4 2 / + 5 * -		1	1 入栈
2 4 2 / + 5 * -	1	2	2 入栈
4 2 / + 5 * -	1 2	4	4 入栈
2 / + 5 * -	1 2 4	2	2 入栈
/ + 5 * -	1 2 4 2	/	2 4 出栈，计算 $4/2=2$ 入栈
+ 5 * -	1 2 2	+	2 2 出栈，计算 $2+2=4$ 入栈
5 * -	1 4	5	5 入栈
* -	1 4 5	*	5 4 出栈，计算 $4*5=20$
-	1 2 0	-	2 0 1 出栈，计算 $1-20=-19$

6、答案：10

解析：根据图的基本性质，一棵树有 n 个结点时，它会有 $n - 1$ 条边。因此，在森林中，如果有 k 棵树，总结点数为 V ，总边数为 E ，那么我们有如下关系：

$E = V - k$ ，其中 E 是边的总数， V 是结点的总数， k 是树的棵数。题目中给定了森林 F 的结点数 $V = 25$ ，边数 $E = 15$ ，我们需要求解树的棵数 $k = 10$ 。

7、答案：(c,d)

解析：这是广义表的考题，对于取表头取表尾的操作可以参考：

例：

$LS = (a, (b, c, d))$

$head(LS) = a$

$tail(LS) = ((b, c, d))$

$head(tail(LS)) = (b, c, d)$

$tail(tail(LS)) = ()$

$head(head(tail(LS))) = b$

$tail(head(tail(LS))) = (c, d)$

对于题目， $getTail[((a, b), (c, d))] = ((c, d))$, $getHead[getTail[((a, b), (c, d))]] = (c, d)$

8、答案：011123

解析：next 数组的求法可以参考王道数据结构串的部分。

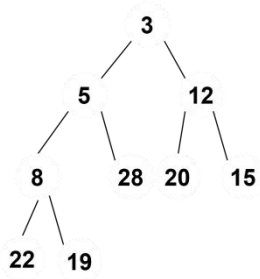
9、答案：82

解析：将二叉树的公式： $n_0 = n_2 + 1$ ，

推广到度为 4： $n_0 = n_2 + 2 * n_3 + 3 * n_4 + 1 = 1 + 2 * 10 + 3 * 20 + 1 = 82$

10

答案：

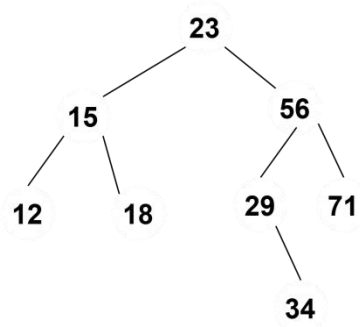


解析：根据堆的调整规则调整即可得到。

三、综合应用题

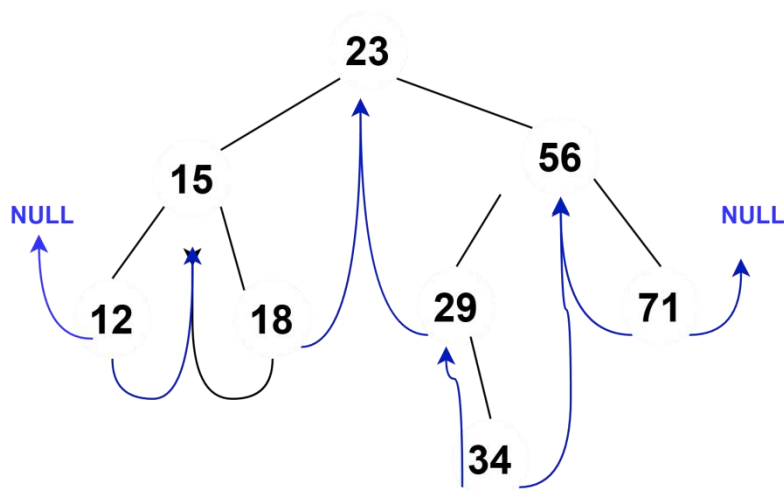
1、答案：

(1)

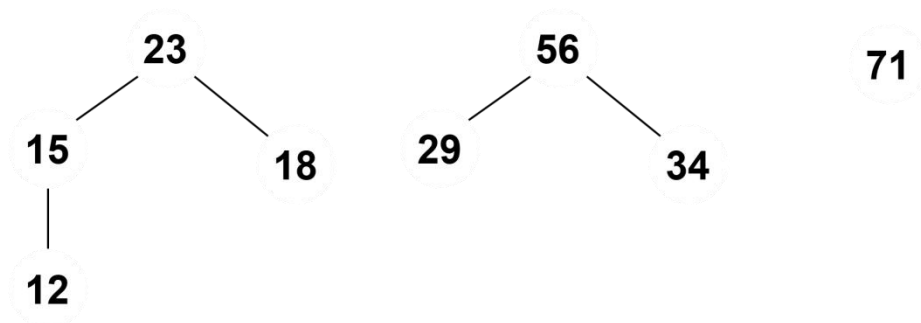


(2) 12,18,15,34,29,71,56,23

(3)



(4)



2、答案：

(1)

哈希地址	0	1	2	3	4	5	6	7	8	9	10	11	12
关键字		59	2		28	31		20	8		25		70
探测次数		2	1		4	1		1	1		4		2

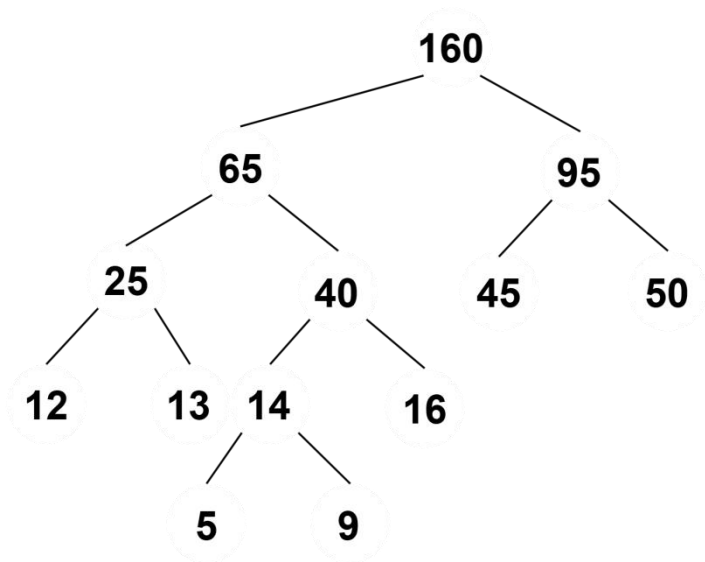
关键字	计算过程
2	$H_0(2)=2\%13=2$ ，成功
8	$H_0(8)=8\%13=8$ ，成功
31	$H_0(31)=31\%13=5$ ，成功
20	$H_0(20)=20\%13=7$ ，成功
70	$H_0(70)=70\%13=5$ ，冲突； $H_1=(H_0+(\text{reverse}(70)+1)\%11+1)\%13=12$ ，成功
59	$H_0(59)=59\%13=7$ ，冲突； $H_1=(H_0+(\text{reverse}(59)+1)\%11+1)\%13=1$ ，成功
25	$H_0(25)=25\%13=12$ ，冲突； $H_1=(H_0+(\text{reverse}(25)+1)\%11+1)\%13=7$ ，冲突； $H_2=(H_1+(\text{reverse}(25)+1)\%11+1)\%13=2$ ，冲突； $H_3=(H_2+(\text{reverse}(25)+1)\%11+1)\%13=10$ ，成功

28	$H_0(28)=28\%13=2$ ，冲突； $H_1=(H_0+(\text{reverse}(28)+1)\%11+1)\%13=7$ ，冲突； $H_2=(H_1+(\text{reverse}(28)+1)\%11+1)\%13=12$ ，冲突； $H_3=(H_2+(\text{reverse}(28)+1)\%11+1)\%13=4$ ，成功
----	--

(2) 查找成功的平均时间复杂度为： $ASL_{\text{成功}} = \frac{1}{8}(2+1+4+1+1+1+4+2) = \frac{16}{8} = 2$

3、答案：

(1)



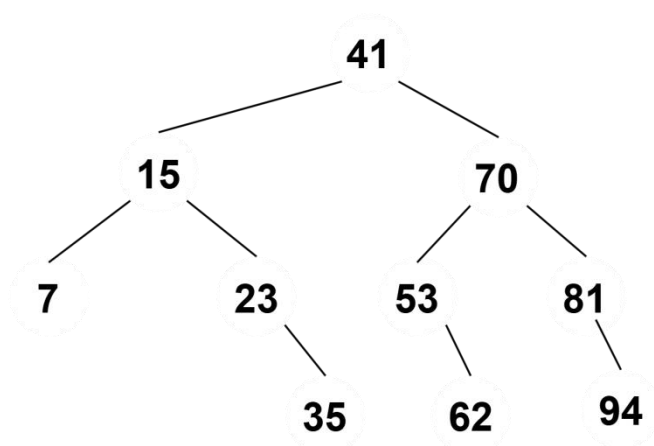
(2)

字符	权值	编码
a	5	0100
b	9	0101
c	12	000
d	13	001
e	16	011

f	45	10
g	50	11

4、答案：

(1)



(2) 41->70->53

(3) 41->70->81->94

(4)

$$ASL_{\text{成功}} = \frac{1}{10}(1 \times 1 + 2 \times 2 + 3 \times 4 + 3 \times 5) = \frac{33}{10}$$

$$ASL_{\text{失败}} = \frac{1}{11}(3 \times 5 + 4 \times 6) = \frac{39}{11}$$

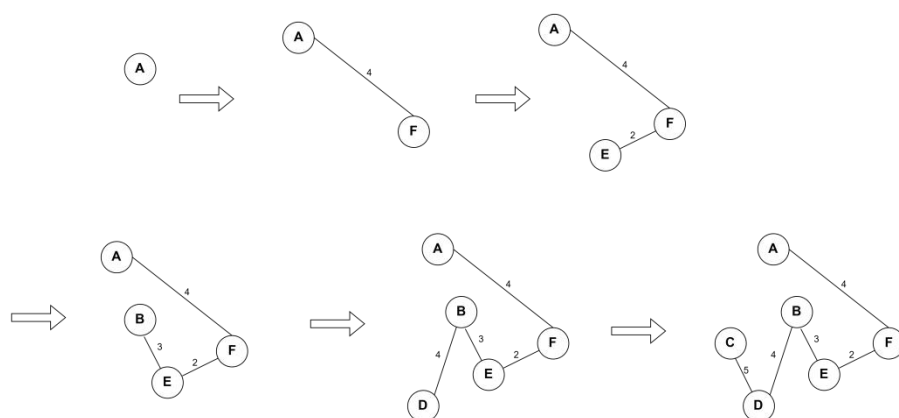
4、答案：

(1)

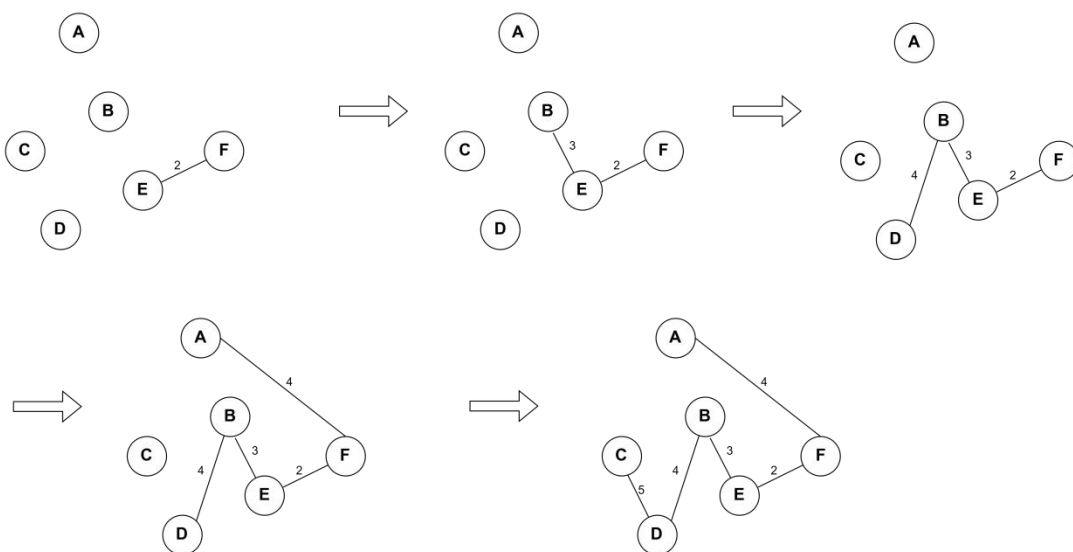
广度优先搜索序列：ABFCDE

深度优先搜索序列：ABCDEF

(2)



(3)



6、答案：

(1)

	a	b	c	d	e	f	h	g	i
ve(Vi)	0	6	4	5	7	16	7	14	19
vl(Vi)	0	6	6	8	7	17	10	14	19

(2)

	ab	ac	ad	be	ce	ef	eg	dh	hg	fi	gi
e(ai)	0	0	0	6	4	7	7	5	7	16	14
l(ai)	0	2	4	6	6	8	7	8	10	17	14
关键活动	√			√			√				√

(3)

关键路径：a->b->e->g->i

四、程序设计题。

答案：

(1) 为了在单链表中找到倒数第 k 个元素，可以使用 **双指针法**（也叫做**快慢指针法**）来高效地解决这个问题。

具体步骤如下：

①初始化两个指针：我们可以使用两个指针，fast 和 slow。初始化时，两个指针都指向链表的头节点。

②移动快指针：先让 fast 指针走 k 步，即 fast 指针比 slow 指针领先 k 个节点。这样，fast 指针和 slow 指针之间的距离是固定的 k。

③同时移动两个指针：从此之后，fast 和 slow 同时移动一步。当 fast 到达链表的末尾时，slow 指针所指向的节点就是倒数第 k 个节点。

④边界条件：如果链表的长度小于 k，则返回 0，表示不存在倒数第 k 个元素。

(2) 代码:

```
1. // 查找倒数第 k 个元素
2. int findKthFromEnd(LinkList head, int k) {
3.     LinkList fast = head;
4.     LinkList slow = head;
5.
6.     // 让 fast 指针先走 k 步
7.     for (int i = 0; i < k; i++) {
8.         if (fast == NULL) {
9.             // 如果链表长度小于 k, 返回 0
10.            return 0;
11.        }
12.        fast = fast->next;
13.    }
14.
15.    // 同时移动 fast 和 slow 指针, 直到 fast 指针到达链表末尾
16.    while (fast != NULL) {
17.        fast = fast->next;
18.        slow = slow->next;
19.    }
20.
21.    // 此时 slow 指针指向倒数第 k 个元素
22.    printf("The %d-th element from the end is: %d\n", k, slow->data);
23.    return 1;
24. }
```

附: 也可以用栈或者递归

2、答案:

(1) 可以使用递归的方法遍历二叉树, 基本思想如下:

①递归遍历: 从根节点开始, 递归地遍历左子树和右子树。对于每一个节点, 计算其左右子树的深度 (即从该节点到达叶子节点的最长路径长度)。

②路径计算: 每个节点的最长路径是通过左子树或右子树的最长路径计算出来的, 我们选择两者中较长的一条路径, 并将当前节点加到路径中。

③路径记录: 在递归过程中, 记录当前路径并更新最长路径。

④终止条件: 递归的终止条件是遇到叶子节点。叶子节点没有子树, 所以返回其路径长度为 1。

⑤结果返回：最终返回的是从根到叶子节点的最长路径，并且路径上所有的节点值。

(2)

```
1. // 计算从根节点到叶子节点的最长路径长度，并记录路径
2. int maxDepth(BiTree root,int& maxLen,int currentLen,int path[],int& pathIndex)
   {
3.     if (root == NULL) {
4.         return 0;
5.     }
6.
7.     // 将当前节点添加到路径中
8.     path[pathIndex] = root->data;
9.     pathIndex++;
10.
11.    // 如果是叶子节点，检查路径长度
12.    if (root->lchild == NULL&& root->rchild == NULL) {
13.        if (currentLen > maxLen) {
14.            maxLen = currentLen;
15.            // 打印当前路径
16.            cout << "Longest path: ";
17.            for (int i = 0; i < pathIndex; i++) {
18.                cout << path[i] << " ";
19.            }
20.            cout << endl;
21.        }
22.    }
23.
24.    // 递归计算左右子树的深度
25.    int leftLen = maxDepth(root->lchild, maxLen, currentLen + 1, path, pathIndex);
26.    int rightLen = maxDepth(root->rchild, maxLen, currentLen + 1, path, pathIndex);
27.
28.    // 回溯，移除当前节点
29.    pathIndex--;
30.
31.    return (leftLen > rightLen) ? leftLen : rightLen;
32. }
```

3、答案：

(1) 为了从顶点 u 到顶点 v 并且经过顶点 k ，我们可以分为两个阶段进行查找：

阶段一：从顶点 u 到顶点 k 。我们使用深度优先搜索（DFS）来查找从顶点 u 到顶点 k 的一条路径。

阶段二：从顶点 k 到顶点 v 。如果阶段一找到了从 u 到 k 的路径，那么接着使用 DFS 查找从 k 到 v 的路径。

(2)

深度优先搜索函数（和参考书有点差异）：

```
1. // 深度优先搜索函数
2. bool DFS(ALGraph *G, int u, int v, int k, int *path, int *pathLength) {
3.     visited[u] = true;
4.     path[*pathLength] = u;
5.     (*pathLength)++;
6.
7.     // 如果找到了从 u 到 k 的路径
8.     if (u == k) {
9.         return true;
10.    }
11.
12.    // 遍历所有邻接的节点
13.    ArcNode *arc = G->vertices[u].first;
14.    while (arc != NULL) {
15.        int w = arc->adjvex;
16.        if (!visited[w]) {
17.            if (DFS(G, w, v, k, path, pathLength)) {
18.                return true; // 找到路径
19.            }
20.        }
21.        arc = arc->next;
22.    }
23.
24.    // 如果没有找到路径，回溯
25.    (*pathLength)--;
26.    visited[u] = false;
27.    return false;
28. }
```

主函数：

```

1. // 主函数，查找路径
2. bool findPath(ALGraph *G, int u, int v, int k) {
3.     int path[MaxVertexNum];
4.     int pathLength = 0;
5.
6.     // 第一步：从 u 到 k
7.     for (int i = 0; i < MaxVertexNum; i++) visited[i] = false; // 初始化访问标志
8.     if (!DFS(G, u, v, k, path, &pathLength)) {
9.         printf("-1\n");
10.        return false; // 找不到从 u 到 k 的路径
11.    }
12.
13.    // 第二步：从 k 到 v
14.    // path[pathLength-1] 是 k，所以从 k 开始继续查找从 k 到 v 的路径
15.    if (path[pathLength-1] != k) {
16.        printf("-1\n");
17.        return false;
18.    }
19.
20.    // 继续进行 DFS 查找从 k 到 v 的路径
21.    for (int i = 0; i < MaxVertexNum; i++) visited[i] = false; // 初始化访问标志
22.    if (!DFS(G, k, v, v, path, &pathLength)) {
23.        printf("-1\n");
24.        return false; // 找不到从 k 到 v 的路径
25.    }
26.
27.    // 输出路径
28.    printf("Path: ");
29.    for (int i = 0; i < pathLength; i++) {
30.        printf("%d ", path[i]);
31.    }
32.    printf("\n");
33.
34.    return true;
35. }

```